Lab 2: Sexism Identification in Twitter

Session 2. Transformers: Fine-tuning for multi-class classification

Goal

The goal of this lab session is to help students understand and gain practice in the use of deep learning-based language models, in particular transformer-based models (BERT, RoBERTa) for English or Spanish. The second objective is the application of these models to text classification tasks, particularly for subtasks 1 and 2 included in the shared task *"EXIST2025: sEXism Identification In Social neTworks"*.

Bibliography

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press.
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. Advances in neural information processing systems, 30.
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT 2019, pages 4171–4186, ACL.
4. Cañete, J., Chaperon, G., Fuentes, R., Ho, J. H., Kang, H., & Pérez, J. (2020). Spanish pre-trained bert model and evaluation data. In Proceeding of the Practical ML for Developing Countries Workshop @ ICLR 2020, pages 1-9, Addis Ababa, Ethiopia.

5. De la Rosa, J., Ponferrada, E., Romero, M., Villegas, P., González de Prado Salas, P., & Grandury, M. (2022). BERTIN: Efficient Pre-Training of a Spanish Language Model using Perplexity Sampling. Procesamiento Del Lenguaje Natural, 68, 13-23
6. Cañete, J., Donoso, S., Bravo-Marquez, F., Carvallo, A., & Araujo, V. (2022). Albeto and distilbeto: Lightweight spanish language models. arXiv preprint arXiv:2204.09145.
7. Gutiérrez-Fandiño, A., Armengol-Estapé, J., Pàmies, M., Llop-Palao, J., Silveira-Ocampo, J., Pio Carrino, C., Armentano-Oller, C., Rodriguez-Penagos, C., Gonzalez-Agirre, A., & Villegas, M. (2022). MarIA: Spanish Language Models. Procesamiento Del Lenguaje Natural, 68, 39-60.
8. https://huggingface.co/docs/transformers/index
9. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Weizhu Chen: LoRA: Low-Rank Adaptation of Large Language Models**.** CoRR abs/2106.09685 (2021)
10. Mangrulkar S., Gugger S., Debut L., and Belkada Y., and Paul S., and Bossa B. (2022). PEFT: State-of-the-art Parameter Efficient Fine-Tuning methods. https://github.com/huggingface/peft.
11. Han, Z., Gao, C., Liu, J., & Zhang, S. Q. (2024). Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*.
12. Ayupov, S., & Chirkova, N. (2022). Parameter-efficient fine tuning of transformers for source code. Workshop on Efficient Natural Language Processing at NeurIPS 2022.

MULTI-CLASS CLASSIFICATION

Multi-class classification is a fundamental machine learning problem where the goal is to assign an input instance to one of several predefined, mutually exclusive categories. It represents a generalization of binary classification, which can be viewed as a special case where only two distinct classes exist. In binary classification, the model determines whether an input belongs to a positive or negative class, typically represented as 0/1 or -1/+1. Multi-class classification extends this paradigm by allowing classification into three or more distinct categories, requiring more sophisticated decision boundaries and predictive strategies. While binary classification involves a simple yes/no or present/absent decision, multi-class problems demand more complex algorithmic approaches such as SoftMax regression, one-vs-rest strategies, or neural network architectures that can simultaneously evaluate multiple class probabilities. The core challenge in multi-class classification lies in developing models that can effectively distinguish between multiple categories based on input features, accounting for the increased complexity of separating instances across more than two potential classes.

FINE TUNING STRATEGIES

Parameter fine-tuning is an optimization technique used to update the weights and biases of a pre-trained model, typically a deep neural network, using a smaller dataset specific to the target task.

The goal is to specialize the model's knowledge and representations learned during the initial pre-training phase to better fit the new task or domain, while leveraging the general features and patterns learned from the larger pre-training dataset. Two of the most common strategies are:

1. **Discriminative Fine-tuning**: This is the most widely used strategy, where the entire pre-trained model, or a subset of its layers, is fine-tuned end-to-end using a supervised learning objective (e.g., cross-entropy loss for classification tasks) on the target task's labeled data.
2. **LoRA (Low Rank Adaptation):** LoRA performs low rank adaptation of the update matrices in transformers to reduce the number of trainable parameters. By decomposing attention weight updates into low-rank matrices, LoRA efficiently fine-tunes transformers with a limited number of parameters.

The choice of fine-tuning strategy depends on factors such as the pre-trained model architecture, the target task, the amount of available labeled data, and the computational resources. In many cases, a combination of these strategies is employed to achieve the best results. It is important to note that parameter fine-tuning is an empirical process often involving extensive hyperparameter tuning, experimentation, and validation to find the optimal configuration for the specific task and dataset. In this session, you learn how to fine-tune a transformer model using **discriminative fine-tuning** and **LoRA** methods.

DISCRIMINATIVE FINE-TUNING

Performing fine-tuning of Transformers

A refined variant involves recalibrating the weights learned by the model based on training examples from the specific task to be solved. To do this, it is necessary to define:

- The Dataset class for data handling
- The function to compute the quality measures
- The training and validation pipeline

After having these elements, a model based on the Transformer can be retrained, followed by performing the predictions on the test dataset.

To fine-tune the transformer's models like BERT or RoBERTa we use the Application Programming Interface (API) provided by the Hugging Face's Transformers library. We will use the *Trainer* and the *TrainingArguments* classes.

The *TrainingArguments* class defines the hyperparameters and other training configurations for the Trainer. It provides a convenient way to specify various training settings without modifying the Trainer class directly. A selection of main hyperparameters is presented below. For a comprehensive list,

please refer to the technical documentation of the TrainingArguments class[1].

Main parameters of *TrainingArguments:*

- *Output_dir*: This parameter specifies the output directory for model checkpoints.
- *Num_train_epochs*: This parameter defines the number of training epochs.
- Learning_rate: This parameter defines the learning rate used to update the weights during the backprop.
- *Per_device_train_batch_size*: This parameter defines the batch size per device during training.
- *Per_device_eval_batch_size*: This parameter defines the batch size per device during evaluation.
- *Warmup_steps*: This parameter defines the number of warmup steps for the optimizer.
- *Weight_decay*: This parameter defines the weight decay for regularization.
- *Logging_dir*: This parameter specifies the directory for logging.
- *Logging_steps*: This parameter indicates Log training metrics every 10 steps.
- *Eval_strategy*: This parameter defines the strategy used to evaluate the model (for example, at the end of each epoch).

[1] https://huggingface.co/docs/transformers/en/main_classes/trainer - transformers.TrainingArguments

- *Save_strategy*: This parameter defines the strategy to save the model (for example at the end of each epoch).
- *Load_best_model_at_end*: This parameter indicates loading the best model at the end of training.
- *Metric_for_best_model*: This parameter defines the measure to use for selecting the best model (for example F1-Score).

The *Trainer* class in Hugging Face's Transformers library is a high-level API that simplifies the training process for several classification tasks. It handles many of the complexities of training, such as:

- **Model Training**: Handles the core training loop, including gradient calculation, parameter updates, and loss minimization.
- **Evaluation**: Evaluates the model on a validation dataset and reports metrics.
- **Checkpoint Saving**: Saves model checkpoints at regular intervals or when a new best model is found.
- **Logging**: Logs training metrics and other information.
- **Early Stopping**: Automatically stops training if the validation performance does not improve for a certain number of epochs.

A selection of the main parameters of the *Trainer* class is presented below. For a comprehensive list, please refer to the technical documentation of the *Trainer* class[2].

[2] https://huggingface.co/docs/transformers/en/main_classes/trainer - transformers.Trainer

Main parameters of Trainer:

- *Model*: The model to be trained.
- *Args*: The instance of *TrainingArguments class* with the specification of the hyperparameters.
- *Train_dataset*: The training dataset (an instance of the Dataset class).
- *Eval_dataset*: The validation dataset (an instance of the Dataset class).
- *Compute_metrics:* The function to compute the evaluation measures.
- *Callbacks*: This parameter specifies a list of callbacks to be used during training. One of the most commonly used callbacks is *EarlyStoppingCallback*, which can be configured to stop training early if the validation performance doesn't improve after a certain number of epochs.

To fine-tune models for addressing the EXIST2025 shared task, specifically subtasks 1 and 2, it is necessary to install all required dependencies for training and validation.

Installing the required dependencies

```
!pip install transformers --upgrade
!pip install datasets accelerate

#Importing the required modules
import os, sys, tempfile, pandas as pd
import numpy as np
```

```
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments, EarlyStoppingCallback
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from sklearn.preprocessing import LabelEncoder
import random
```

In a second step it is required to extend the torch.Dataset class to handle the data of the EXIST2025 dataset for subtasks 1 and 2.

EXAMPLE 1: SeximsDataset extends Dataset torch class

```
class SexismDataset(Dataset):
   def __init__(self, texts, labels, ids, tokenizer, max_len=128, pad="max_length",
trunc=True,rt='pt'):
       self.texts = texts.tolist()
       self.labels = labels
       self.ids = ids
       self.tokenizer = tokenizer
       self.max_len = max_len
       self.pad = pad
       self.trunc = trunc
       self.rt = rt


   def __len__(self):
       return len(self.texts)

   def __getitem__(self, idx):
       text = str(self.texts[idx])
       inputs = self.tokenizer.encode_plus(
           text,
           add_special_tokens=True,
           max_length=self.max_len,padding=self.pad, truncation=self.trunc,
           return_tensors=self.rt
       )

       return {
           'input_ids': inputs['input_ids'].flatten(),
           'attention_mask': inputs['attention_mask'].flatten(),
           'labels': torch.tensor(self.labels[idx], dtype=torch.long),
           'id': torch.tensor(self.ids[idx], dtype=torch.long)
       }
```

After creating the *SexismDataset* class to handle EXIST2025 data, it is necessary to define evaluation metrics for training and validation. The EXIST2025 shared task requires specific performance measures for each subtask: i) F1-score of the positive

class for subtask1 ii) macro-average F1-score for subtask 2. Examples 2 and 3 illustrate how to implement custom metric computation functions for use with the *Trainer* object during training and validation processes.

EXAMPLE 2: Measure for the subtask 1

```
def compute_metrics_1(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='binary'
    )
    acc = accuracy_score(labels, preds)
    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall
    }
```

EXAMPLE 3: Measure for subtask 2

```
def compute_metrics_2(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='macro'
    )
    acc = accuracy_score(labels, preds)

    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall    }
```

In the Transformer model fine-tuning with HuggingFace, two critical classes facilitate the process:

- *AutoTokenizer class* dynamically selects and loads the appropriate tokenization method for a specific pre-trained

model, ensuring precise text preprocessing aligned with the model's original training.

- *AutoModelForSequenceClassification* enables seamless adaptation of transformer models for classification tasks by providing a flexible framework for loading pre-trained models with custom configurations. For initializing AutoModelForSequenceClassification, three fundamental parameters drive the model's customization. The *model*_name: which specifies the base pre-trained model (e.g., 'bert-base-uncased'). The parameter *num_labels which d*efines the number of classification categories in your specific task. The parameter *problem_type* which indicates the classification approach (e.g., 'single_label_classification').

Example 4 integrates the classes *SeximsDataset, AutoTokenizer, AutoModelForSequenceClassification*, *TrainingArguments* and *Trainer* for training and validating a RoBERTa model for sexism classification in English. The workflow encompasses:

- Training the model using the training dataset.
- Evaluating performance on the validation set.
- Generating predictions for the test set.

Key steps include model configuration, tokenization, training with predefined parameters, and running inference to obtain classification results.

Note that to fine-tune other models based on the BERT architecture, such as BERT, RoBERTa, or BERT-based models for specific languages like BETO for Spanish, you simply need to change the *model_name* parameter. It makes it easy to experiment with different models.

EXAMPLE 4: Training and Validation pipeline for subtask1

```
def sexism_classification_pipeline_task1(trainInfo, devInfo, testInfo=None,
model_name='roberta-base', nlabels=2, ptype="single_label_classification", **args):
    # Model and Tokenizer
    labelEnc= LabelEncoder()
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=nlabels,
        problem_type=ptype
    )

    # Prepare datasets
    train_dataset = SexismDataset(trainInfo[1], labelEnc.fit_transform(trainInfo[2]),[int(x)
for x in trainInfo[0]], tokenizer )
    val_dataset = SexismDataset(devInfo[1], labelEnc.transform(devInfo[2]), [int(x) for x in
devInfo[0]], tokenizer)

    # Training Arguments
    training_args = TrainingArguments(
        report_to="none", # alt: "wandb", "tensorboard" "comet_ml" "mlflow" "clearml"
        output_dir= args.get('output_dir', './results'),
        num_train_epochs= args.get('num_train_epochs', 5),
        learning_rate=args.get('learning_rate', 5e-5),
        per_device_train_batch_size=args.get('per_device_train_batch_size', 16),
        per_device_eval_batch_size=args.get('per_device_eval_batch_size', 64),
        warmup_steps=args.get('warmup_steps', 500),
        weight_decay=args.get('weight_decay',0.01),
        logging_dir=args.get('logging_dir', './logs'),
        logging_steps=args.get('logging_steps', 10),
        eval_strategy=args.get('eval_strategy','epoch'),
        save_strategy=args.get('save_strategy', "epoch"),
        load_best_model_at_end=args.get('load_best_model_at_end', True),
        metric_for_best_model=args.get('metric_for_best_model',"f1")
    )

    # Initialize Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        compute_metrics=compute_metrics_1,

callbacks=[EarlyStoppingCallback(early_stopping_patience=args.get("early_stopping_patience",3))
]
    )
```

```
    # Fine-tune the model
    trainer.train()

    # Evaluate on validation set
    eval_results = trainer.evaluate()
    print("Validation Results:", eval_results)

    # If there is a test dataset
    if testInfo is not None:
        # Prepare test dataset for prediction
        test_dataset = SexismDataset(testInfo[1], [0] * len(testInfo[1]),  [int(x) for x in
testInfo[0]],   tokenizer)

        # Predict test set labels
        predictions = trainer.predict(test_dataset)
        predicted_labels = np.argmax(predictions.predictions, axis=1)

        # Create submission DataFrame
        submission_df = pd.DataFrame({
            'id': testInfo[0],
            'label': labelEnc.inverse_transform(predicted_labels),
            "test_case": ["EXIST2025"]*len(predicted_labels)
        })
        submission_df.to_csv('sexism_predictions_task1.csv', index=False)
        print("Prediction for TASK 1 completed. Results saved to sexism_predictions_task1.csv")
        return model, submission_df
    return model, eval_results
```

Example 5 illustrates how the BERT model can be fine-tuned for subtask 2 that consists in a multi-class (three classes) classification problem.

EXAMPLE 5: Training and validation pipeline for subtask2

```
def sexism_classification_pipeline_task2(trainInfo, devInfo, testInfo=None, model_name='bert-
base-uncased', nlabels=3, ptype="single_label_classification", **args):
    # Model and Tokenizer
    labelEnc= LabelEncoder()
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=nlabels,
        problem_type=ptype
    )

    # Prepare datasets
    train_dataset = SexismDataset(trainInfo[1], labelEnc.fit_transform(trainInfo[2]),[int(x)
for x in trainInfo[0]], tokenizer )
    val_dataset = SexismDataset(devInfo[1], labelEnc.transform(devInfo[2]), [int(x) for x in
devInfo[0]], tokenizer)

    # Training Arguments
    training_args = TrainingArguments(
        report_to="none", # alt: "wandb", "tensorboard" "comet_ml" "mlflow" "clearml"
        output_dir= args.get('output_dir', './results'),
```

```
        num_train_epochs= args.get('num_train_epochs', 5),
        learning_rate=args.get('learning_rate', 5e-5),
        per_device_train_batch_size=args.get('per_device_train_batch_size', 16),
        per_device_eval_batch_size=args.get('per_device_eval_batch_size', 64),
        warmup_steps=args.get('warmup_steps', 500),
        weight_decay=args.get('weight_decay',0.01),
        logging_dir=args.get('logging_dir', './logs'),
        logging_steps=args.get('logging_steps', 10),
        eval_strategy=args.get('eval_strategy','epoch'),
        save_strategy=args.get('save_strategy', "epoch"),
        save_total_limit=args.get('save_total_limit', 1),
        load_best_model_at_end=args.get('load_best_model_at_end', True),
        #metric_for_best_model=args.get('metric_for_best_model',"ICM")
        metric_for_best_model=args.get('metric_for_best_model',"f1")
    )

    # Initialize Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        compute_metrics=compute_metrics_2,

callbacks=[EarlyStoppingCallback(early_stopping_patience=args.get("early_stopping_patience",3))
]
    )

    # Fine-tune the model
    trainer.train()

    # Evaluate on validation set
    eval_results = trainer.evaluate()
    print("Validation Results:", eval_results)

    # If there is a test dataset
    if testInfo is not None:
        # Prepare test dataset for prediction
        test_dataset = SexismDataset(testInfo[1], [0] * len(testInfo[1]),  [int(x) for x in
testInfo[0]],   tokenizer)

        # Predict test set labels
        predictions = trainer.predict(test_dataset)
        predicted_labels = np.argmax(predictions.predictions, axis=1)

        # Create submission DataFrame
        submission_df = pd.DataFrame({
            'id': testInfo[0],
            'label': labelEnc.inverse_transform(predicted_labels),
            "test_case": ["EXIST2025"]*len(predicted_labels)

        })
        submission_df.to_csv('sexism_predictions_task2.csv', index=False)
        print("Prediction TASK2 completed. Results saved to sexism_predictions_task2.csv")
        return model, submission_df
    return model, eval_results
```

Example 6 Illustrates the dataset loading process for both subtasks and illustrates the model fine-tuning approach for EXIST2025 subtasks 1 and 2.

EXAMPLE 6: Addressing subtasks 1 and 2

```
from readerEXIST2024 import EXISTReader
# path to the dataset, adapt this path wherever you have the dataset
dataset_path = os.path.join(base_path, "EXIST_2025_Dataset_V0.2/")

file_train = os.path.join(dataset_path, "EXIST2025_training.json")
file_dev = os.path.join(dataset_path, "EXIST2025_dev.json")

reader_train = EXISTReader(file_train)
reader_dev = EXISTReader(file_dev)

# Reading the Training, Development and Test sets for subtask1 and subtask2 in English
EnTrainTask1, EnDevTask1 = reader_train.get(lang="EN", subtask="1"), reader_dev.get(lang="EN",
subtask="1")
EnTrainTask2, EnDevTask2 = reader_train.get(lang="EN", subtask="2"), reader_dev.get(lang="EN",
subtask="2")


# Here it is possible to define some hyperparameters used during the training and validation
phases
set_seed()
# modelname
modelname = "cardiffnlp/twitter-roberta-base-2022-154m"
# modelname = "roberta-base"
# training params
params = {
    "num_train_epochs": 10
}

# pipeline for the subtask1
m1, res_en1 = sexism_classification_pipeline_task1(EnTrainTask1, EnDevTask1, None, modelname,
2, "single_label_classification", **params )

# pipeline for the subtask2
m2, res_en2 = sexism_classification_pipeline_task2(EnTrainTask2, EnDevTask2, None,  modelname,
3, "single_label_classification", **params )
```

LOW-RANK ADAPTATION

To perform the LoRA method, you can use the implementation provided in the Parameter-Efficient Fine-Tuning (PEFT)[3] python library. PEFT is a library developed by Hugging Face that enables efficient fine-tuning of Transformers models, such as BERT, RoBERTa, etc., for various downstream tasks.

The key idea behind PEFT is to introduce a small number of trainable parameters while keeping most of the pre-trained model's parameters frozen. This approach significantly reduces the memory footprint and computational requirements during fine-tuning, making it feasible to fine-tune large language models on resource-constrained devices or with limited computational resources.

PEFT provides several techniques for parameter-efficient fine-tuning, including:

- LoRA which introduces low-rank matrices to modify the weights of the pre-trained model during fine-tuning, as can be observed in Fig 1. These low-rank matrices are much smaller than the original model weights, significantly reducing the number of trainable parameters.

[3] https://github.com/huggingface/peft

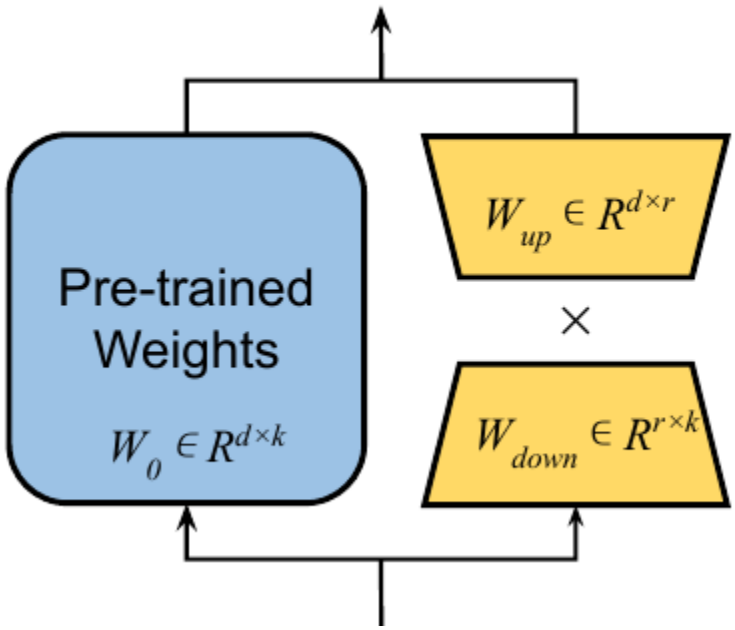

Figure 1: The main idea behind the Low Rank-Adaptation

Using PEFT you can fine-tune Transformers models with minimal additional storage and computational requirements, making it easier to deploy these models in various scenarios, including edge devices and resource-constrained environments. The library is designed to work seamlessly with the Hugging Face Transformers library, providing a user-friendly interface for applying parameter-efficient fine-tuning techniques to pre-trained models from the Hugging Face model hub.

Performing Parameter Efficient Fine Tuning of Transformers. Low Rank Adaptation.

A refined variant involves recalibrating the weights learned by the model based on training examples from the specific task to be

solved. This time, the weights of the original model are kept frozen, whereas additional weights are learned, considering the pretrained weights. To perform LoRA integration it is necessary to install the PEFT library and import the required modules.

Installing PEFT library

```
!pip install peft
from peft import LoraConfig, get_peft_model, TaskType
```

After that, it is required to perform the following steps:

- Pick up the base model to be used, which includes a head for sequence classification.
- The setting of the LoRA parameters, an introduction of the new parameters (weights) to the model.
- Incorporate the LoRA matrices to the model
- Save, load, and mix the new weights learned with the weights of the base models.

After having these elements, the model based on Transformers can be retrained, followed by performing the predictions on the test dataset. Example 7 illustrates the changes in the pipeline, which are quite simple.

EXAMPLE 7: Fine-Tuning RoBERTa model for subtask 1 using LoRA

```
#########################################CHANGE#################################################
from peft import LoraConfig, get_peft_model, TaskType
#################################################################################################
def sexism_classification_pipeline_task1_LoRA(trainInfo, devInfo, testInfo, model_name='roberta-base', nlabels=2,
ptype="single_label_classification", **args):
```

```
# Model and Tokenizer
labelEnc= LabelEncoder()
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
model_name,
num_labels=nlabels,
problem_type=ptype

)
#######################################CHANGE#################################################
# Configure LoRA
lora_config = LoraConfig(
task_type= args.get("task_type" TaskType.SEQ_CLS),
target_modules= args.get("target_modules", ["query", "value"]),
r= args.get("rank", 64), # Rank of LoRA adaptation
lora_alpha=args.get("lora_alpha",32), # Scaling factor
lora_dropout=args.get("lora_dropout",0.1),
bias=args.get("bias","none")
)
##########################################################################################

#######################################CHANGE#################################################
# Prepare LoRA model
peft_model = get_peft_model(model, lora_config)
##########################################################################################
# Prepare datasets
train_dataset = SexismDataset(trainInfo[1], labelEnc.fit_transform(trainInfo[2]),[int(x) for x in trainInfo[0]], tokenizer )
val_dataset = SexismDataset(devInfo[1], labelEnc.transform(devInfo[2]), [int(x) for x in devInfo[0]], tokenizer)

# Training Arguments
training_args = TrainingArguments(
output_dir= args.get('output_dir', './results_task1_LoRA0'),
num_train_epochs= args.get('num_train_epochs', 5),
learning_rate=args.get('learning_rate', 5e-5),
per_device_train_batch_size=args.get('per_device_train_batch_size', 16),
per_device_eval_batch_size=args.get('per_device_eval_batch_size', 64),
warmup_steps=args.get('warmup_steps', 500),
weight_decay=args.get('weight_decay',0.01),
logging_dir=args.get('logging_dir', './logs'),
logging_steps=args.get('logging_steps', 10),
eval_strategy=args.get('eval_strategy','epoch'),
save_strategy=args.get('save_strategy', "epoch"),
save_total_limit=args.get('save_total_limit', 1),
load_best_model_at_end=args.get('load_best_model_at_end', True),
metric_for_best_model=args.get('metric_for_best_model","f1")
)

# Initialize Trainer
```

```
trainer = Trainer(
########################################CHANGE#################################################
# Prepare LoRA model
model=peft_model,
#########################################################################################
args=training_args,
train_dataset=train_dataset,
eval_dataset=val_dataset,
compute_metrics=compute_metrics_1,
callbacks=[EarlyStoppingCallback(early_stopping_patience=args.get("early_stopping_patience",3))]
)

# Fine-tune the model
trainer.train()

# Evaluate on validation set
eval_results = trainer.evaluate()
print("Validation Results:", eval_results)

########################################CHANGE#################################################
#Saving the new weigths for the LoRA model
trainer.save_model('./final_best_model_LoRA')
# Notice that, in this case only the LoRA matrices are saved.
# The weigths for the classificaction head are not saved.
#########################################################################################
########################################CHANGE#################################################
#Mixing the LoRA matrices with the weigths of the base model used
mixModel=petf_model.merge_and_unload()
mixModel.save_pretrained('./final_bestfullmodel')
# IN this case the full model is saved.
#########################################################################################

if testInfo is not None:
        # Prepare test dataset for prediction
        test_dataset = SexismDataset(testInfo[1], [0] * len(testInfo[1]),  [int(x) for x in
testInfo[0]],   tokenizer)

        # Predict test set labels
        predictions = trainer.predict(test_dataset)
        predicted_labels = np.argmax(predictions.predictions, axis=1)

        # Create submission DataFrame
        submission_df = pd.DataFrame({
            'id': testInfo[0],
            'label': labelEnc.inverse_transform(predicted_labels),
            "test_case": ["EXIST2025"]*len(predicted_labels)
        })
        submission_df.to_csv('sexism_predictions_task1.csv', index=False)
        print("Prediction for TASK 1 completed. Results saved to sexism_predictions_task1.csv")
        return model, submission_df
    return model, eval_results
```

ACTIVITY

The activity consists of developing and fine-tuning Transformer-based models for the "*EXIST2025: sEXism Identification In Social neTworks*" shared task. Students must address subtasks 1 and 2. They can use the Transformer-based models studied in the lab session (BERT and RoBERTa) or any other models. For fine-tuning the models, the techniques studied in the lab sessions must be used. Developing and training your models on hardware equipped with a GPU (Graphics Processing Unit) is crucial to leverage the computational power and accelerate the training process.

Deliver **the source** code and include **a comprehensive report explaining your approach**, strategy, and findings. Include the following aspects:

- Data preprocessing steps, if any.
- Model selection and architecture details.
- Fine-tuning techniques and hyperparameter selection strategies.
- Evaluation measures and results.
- Relevant visualizations and tables to support your work.

To achieve the maximum score, two models must be trained with two different parameter configurations for the two subtasks, the two languages, and the two techniques (fine-tuning and LoRA), to obtain at least 16 results = 2 (model+param) x 2 (subtasks) x 2 (techniques) x 2 (languages) and make a **good** report.

The Google collaborative environment (Google Colab) can be used to fine-tune the Transformer models: https://colab.research.google.com/notebooks/welcome.ipynb#scrollTo=5fCEDCU_qrC0

The delivery of this assignment will be carried out jointly with the assignment of the next session lab2-s3.

APPENDIX

The set_seed() function is critical for ensuring reproducibility in deep-learning experiments by establishing a consistent random number generation environment across different computational resources.

```
def set_seed(seed=1234):
    """
    Sets the seed to make everything deterministic, for reproducibility of experiments
    Parameters:
    seed: the number to set the seed to
    Return: None
    """
    # Random seed
    random.seed(seed)
    # Numpy seed
    np.random.seed(seed)
    # Torch seed
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = True
    # os seed
    os.environ['PYTHONHASHSEED'] = str(seed)
```